Solving Job-Shop Scheduling Via Uninformed Search, Heuristic Search, Local Search, and Mixed Integer Linear Programming

Juan José H. Beltrán[1]
[1]Tecnologico de Monterrey, Escuela de Ingeniería y Ciencias.

October 25, 2026

1 Introduction

An optimization problem involves finding the best feasible solution from a set of possible solutions for a problem at hand, maximizing or minimizing a fitness function by searching and selecting its best values (Valencia-Rivera et al. 2024). Multiple approaches, such as uniformed search (US), heuristic search (HS), local search (LS), constraint satisfaction methods (CSP), just to mention some examples, have been developed to deal with this problems. As we can expect, consequence of the No Free Lunch Theorem (Wolpert and Macready 1997), each approach has its own advantages and weaknesses over distinct problem domains.

The job-shop scheduling problem (JSSP) is an optimization problem where some resources (commonly referred as machines) must be allocated to some tasks (referred as jobs) trying to minimize the total completion time for all the tasks (a.k.a. the makespan). Each job consists of some operations that must be performed in a fixed order, and each operation must be processed on a specific machine for a given processing time.

The objective of this work is to compare the performance of different algorithms (of the US, HS, and LS types), as well as different parameter configurations of them, applied to multiple instances of JSSP, restricted to a time limit. We will measure the performance using two metrics: The *relative gap* between the theoretical optimal value and the value obtained by practical algorithms, and the *computation time* per instance measured in seconds, on equal terms.

1.1 Problem Statement

In its classical form, the Job-Shop Scheduling Problem is defined as follows (Xiong et al. 2022): A job-shop environment contains several machines $M = \{M_1, M_2, ..., M_m\}$, and several jobs $J = \{J_1, J_2, ..., J_n\}$. Each job $J_j \in J$ contains a serial of operations $O_j = \{O_{j1}, O_{j2}, ...O_{jm}\}$ which need to be processed in a predefined sequence. Each operation $O_{jk} \in O_j$ for some job $J_j \in J$ is assigned to a specific machine $M_i \in M$ to be processed with a given processing time p_{jk}. Sequencing need to be done for all operations in their corresponding machines to minimize the maximum completing time of all jobs, i.e., to minimize the makespan. It is common to assume that each job has exactly one operation per machine, so in an instance with m machines and n jobs, each job also has m sequential operations.

In order to preserve feasibility in real-world applications we consider three constraints:

(a) Precedence: Each operation of a job must not start before the its previous operation is completed.

(b) Machine capacity: Each machine processes at most one operation at a time (with no overlap).

(c) No preemption: Once started, each operation runs to completion.

It is worth to mention that one of the most common ways to model the JSSP is the disjunctive model (see Google 2015 and Balas 1969), that uses a disjunctive graph to represent the restrictions mentioned above and transforms the problem into finding a graph construction that minimizes the the longest weighted path between two nodes (called the *critical path*). But, as it is not fundamental for this work, I will not go into further detail.

1.2 Linear Mathematical Model

As explained by Manne (1960) (see Özgüven, Özbakır, and Yavuz 2010 as well) the classic JSSP can be represented as a mixed linear integer programming (MILP) problem through the following model.

1.2.1 Variables

Let $s_{jk} \in \mathbb{R}^+ \cup 0 \ \ \forall j \in J, k \in O_j$ be the start time of the operation O_{jk}. The boolean variables $y_{j1,k1,j2,k2}$ for each pair of operations $(O_{j1,k1}, O_{j2,k2})$ assigned to the same machine, takes the value of 1 if $O_{j2,k2}$ is processed after $O_{j1,k1}$, and takes the value of 0 otherwise. Finally C_{max} corresponds to the makespan of the schedule.

1.2.2 Parameters

Let $p_{jk} \in \mathbb{R}^+ \cup 0 \ \ \forall j \in J, k \in O_j$ be the given processing time of the operation O_{jk}. Also let $\mathcal{M}$ be a large number (according to the big $\mathcal{M}$ method).

1.2.3 Objective Function

$$\min C_{max}$$

1.2.4 Constraints

$$s_{j,k+1} \geq s_{jk} + p_{jk} \qquad \forall j \in J, k \in O_j \setminus \{O_{jm}\} \tag{1}$$

$$s_{j2,k2} \geq s_{j1,k1} + p_{j1,k1} - \mathcal{M} * (1 - y_{j1,k1,j2,k2}) \tag{2}$$

for each pair of operations $(O_{j1,k1}, O_{j2,k2})$ assigned to the same machine.

$$s_{j1,k1} \geq s_{j2,k2} + p_{j2,k2} - \mathcal{M} * y_{j1,k1,j2,k2} \tag{3}$$

for each pair of operations $(O_{j1,k1}, O_{j2,k2})$ assigned to the same machine.

$$C_{max} \geq s_{jk} + p_{jk} \qquad \forall j \in J, k \in O_j \tag{4}$$

$$s_{j1,k1} \geq 0 \qquad \forall j \in J, k \in O_j \tag{5}$$

$$y_{j1,k1,j2,k2} \in \{0, 1\} \tag{6}$$

for each pair of operations $(O_{j1,k1}, O_{j2,k2})$ assigned to the same machine.

$$C_{max} \geq 0 \tag{7}$$

Equation 1 corresponds to the precedence constraint (see 1.2(a)). Equations 2 and 3 model the machine capacity constraint (see 1.2(b)). Equation 4 ensures that the makespan corresponds to the maximum completing time of all jobs. Finally, equations 5, 6 and 7 model the nature of the variables.

2 Methodology

Given a set of JSSP own generated and benchmark instances (see Section 2.1), and a set of algorithms, some with different parameter configurations, each algorithm will be executed on each instance. Each execution will be limited to 60 seconds, using the same hardware and operative system and software, as specified in Appendix C. If the time limit is reached, the algorithms will stop, report and save the best feasible solution they have found (if they find one). The minimal makespan found and the runtime in seconds will be logged for each execution, as well as other interesting metrics specific to each algorithm. All the algorithm implementations consider a base makespan lower-bound of $\sum p_{jk}$ for all operations in the instance. The tested algorithms can be classified into the following categories:

(a) Uninformed search: Depth-first search (see Sec. 2.2.1), and uniform cost search (see Sec. 2.2.2).

(b) Heuristic search: Weighted A*, and beam search (see Section 2.3).

(c) Local search: Simulated annealing (see Section 2.4).

A constraint satisfaction problem approach, based on the linear model described in Section 1.2, was used to solve each own generated instance to optimality (see Section 2.5), so the *relative gap* to the optimal makespan can be computed for each execution using the Equation 8, where $f(x^*)$ is the optimal value of the objective function known for the given instance, and $f(\hat{x})$ is the value of the objective function obtained in each execution. Thus, the gap can be interpreted as "the proposed solution is $gap\%$ above the optimum". The smaller the gap, the better the proposed solution.

$$\textbf{Gap}(f(\hat{x}), f(x^*)) = \frac{f(\hat{x}) - f(x^*)}{f(x^*)} \tag{8}$$

2.1 Test Instances

In the aggregate, 83 JSSP instances where collected to test the algorithms (see Table 1). 25 of them where generated for the purposes of this work using the Algorithm 1, with an intended distribution over difficulty levels (see Appendix B). This generated instances are labeled as *jb* and numbered from the 1 to the 25. Additionally, 15 more instances with higher difficult levels were generated (and can be found in the repository of Appendix A, as well as the results associated with them), but as they cannot be solved to optimality in reasonable times, they were excluded of this work in order to have a fair comparison across all instances.

Algorithm 1 GENERATE JSSP INSTANCE

Require: n (number of jobs), m (number of machines), $d_{\min} = 1$, $d_{\max} = 99$ (range of processing times), seed (random seed).
Ensure: Job list J, where each job contains the tuple (machine, duration).

if seed **then**
    RANDOMSEED(seed)
$J \leftarrow []$ ▷ Initialize the instance structure.
for $i \leftarrow 1$ a N_{jobs} **do**
    $machines \leftarrow [0, 1, \ldots, N_{\text{machines}} - 1]$
    SHUFFLE($machines$) ▷ Random permutation of machines.
    $job_tasks \leftarrow []$
    for cada $m \in machines$ **do**
        $duration \leftarrow$ RANDOMINT($d_{\min}, d_{\max}$) ▷ Fix a random processing time $\in [d_{\min}, d_{\max}]$.
        APPEND(job_tasks, $(m, duration)$)
    APPEND(J, job_tasks)
return J

As well 43 instances were retrieved from the JSPLib (ScheduleOpt 2024) compilation, namely 3 instances belong to the work of Fisher and Thompson 1963 (labeled as *ft06*, *10* and *20* instances), and 40 instances belong to the work of Lawrence 1984 (labeled as *la* instances, from 1 to 40). This compilation of benchmark instances also stores the known optima, so it was not necessary to solve them via MILP.

Table 1: Distribution of instances over sets and difficulties.

Source	**Toy**	**Easy**	**Medium**	**Hard**	**Total**
Fisher-Thompson 1963 (*ft*)	0	2	0	1	3
Lawrence 1984 (*la*)	0	10	15	15	40
Own generated (*jb*)	5	20	0	0	25
All sources	5	32	15	16	68

All instances, algorithms and experiment results are stored in the GitHub repository (see Appendix A). Each instance was stored in a *txt* file using the standard JSSP format (Google 2015 has a good description of it). The solutions obtained by each execution are stored in *txt* files with one line per

operation. The logs (with the best makespan found and the runtime in seconds) are stored in a *csv* file with one row per execution. All the code is written in Python.

2.2 Uninformed Search

Uninformed search algorithms are designed to explore a tree structure, trying to find the minimum weighted path length from the root to a leave. JSSP solution construction can be modeled as a tree if the root is an empty schedule, and each child is their parents schedule with just one additional feasible operation scheduled (as soon as the machine availabilities allow it). Thus, each leave is a possible feasible schedule with all the operations scheduled.

Given node $\mathcal{N} = \{\psi_{x1,y1}, \psi_{x2,y2}, ..., \psi_{y\tau,x\tau}\}$, each ordered element $\psi_{x,y}$ represents that the operation $O_{x,y}$ has been scheduled in the earliest possible time after scheduling the previous elements following the same rule. Equation 9 formalizes the set of new nodes reached when node $\mathcal{N}$ is expanded. The cost of moving to a child node $\mathcal{N}'$ from its parent $\mathcal{N}$ is the processing time $p_{y\tau,x\tau}$ of the additional scheduled operation.

$$\mathbf{Exp}(\mathcal{N}) = \{\mathcal{N} \cup \psi_{j,k} \mid J_j \in J, k = \max(\{y \mid \psi_{j,y} \in \mathcal{N}\}) + 1\} \tag{9}$$

Said that, it is easy to approximate the branching factor of this tree as n (the number of jobs) for almost every level, as you can always expand adding the next operation of any of the uncompleted jobs to the current schedule. The depth of the tree is $j * m$ (the number of operations to schedule) for all its branches. It is clear that for non-*toy* instances, reaching a leave (a complete schedule) exploring the tree in width will be impossible with a 60 seconds time limit.

2.2.1 Depth-First Search (DFS)

DFS was implemented following the previous modelation. This algorithm starts from the root and always expands the deepest nodes first, so it reaches a complete schedule in just a few expansions, guaranteeing at least one feasible solution within the time limit.

2.2.2 Uniform Cost Search (UCS)

Knowing that a pure Breadth-First Search (DFS) will have trouble reaching a complete schedule within the time limit, a UCS was implemented, expecting that its preference for lower cost paths will improve the DFS performance.

2.3 Heuristic Search

The heuristic search approach follows exactly the same tree modeling used for uninformed search (see Section 2.2), but now taking advantages of heuristics to guide the search. Two heuristic search methods were implemented: Weighted A* and Beam Search. As A* tends to search in width, even if a big weight w attenuates this behavior, Beam Search limits the frontier, so the algorithm can reach some leaves within the time limit. Two heuristic functions (one of them intentionally more informative) were implemented in order to guide the search (described in the following sections).

Weigthed A* was executed over all instances combining two different weight values ($w = 1$, and $w = 100$), and the two possible heuristics. Beam Search was executed over all instances, using just the most informative heuristic, but varying over to different beam widths ($bw = 100$, and $bw = 1000$).

2.3.1 Combined Bound Heuristic (CoBo)

It is based it two lower-bounds (so it is admissible): The maximum sum of operation processing times not scheduled yet per machine (see Equation 10); and the maximum sum of operation processing times not scheduled yet per job (see Equation 11). CoBo takes the max of the two lower-bounds.

$$\mathrm{Lb}_{\mathrm{machine}}(\mathcal{N}) = \max\left(\left\{\sum_{\psi_{jk} \in \{\text{Ops. to perform in M}\}\setminus\mathcal{N}} p_{jk} \;\middle|\; M_i \in M\right\}\right) \tag{10}$$

$$\mathrm{Lb}_{\mathrm{job}}(\mathcal{N}) = \max\left(\left\{\sum_{\psi_{jk} \in O_j \setminus \mathcal{N}} p_{jk} \;\middle|\; j \in J\right\}\right) \tag{11}$$

$$h_{\text{CoBo}}(\mathcal{N}) = \max(\text{Lb}_{\text{machine}}(\mathcal{N}), \text{Lb}_{\text{job}}(\mathcal{N})) \tag{12}$$

2.3.2 Most Work Remaining on Machine Heuristic (MWRM)

Computes the maximum procesing time remaining among all machines, so it prioritizes assigning operations to the machine with most remaining work. As it is a lower bound, it is also admissible. Its complexity is also $O(n * m)$, but its clear that is easier to compute than CoBo.

$$h_{\text{MWRM}}(\mathcal{N}) = \text{Lb}_{\text{machine}}(\mathcal{N}) \tag{13}$$

2.4 Local Search

For local search we consider the solution space as the set of all completed schedules. Each completed schedule $\mathcal{S}$ can be represented as a permutation of the multiset $\Pi = \bigcup_1^m \{1, 2, ..., n\}$, where the k-th appearance of $j \in \mathcal{S}$ means that the operation O_{jk} must be scheduled as early as it does not violate any constraints (i.e. ψ_{jk}). Thus, $\mathcal{S}$ can be decoded in a similar way as we stated in Section 2.2, where each element, from the first to the last, is first scheduled. An initial feasible solution can be generated just as a random permutation of Π, as any permutation of Π can be decoded into a single feasible solution. Also, it can be proven that the optimal schedule is the result of the decoding of some permutation of Π.

In order to define a notion of locality to explore the search space, we introduce neighborhood operators that, applied to a permutation $\mathcal{S}$, return a feasible neighbor. They are described in the following section.

For this approach, simulated annealing was implemented. The algorithm was executed over all instances using three different neighborhood operators, and two different *cooling ratios* for each one.

2.4.1 Operators

As said before, a state $\mathcal{S}$ is represented as a permutation $\mathcal{S} = (\pi_1, \pi_2, \ldots, \pi_{n*m})$ of length $n*m$, containing all elements in Π. To generate neighbor solutions $S' \in \mathbf{N}(S)$, three stochastic operators were implemented:

(a) **Random Swap Operator (RSO).** Selects two distinct positions $i, j \in \{1, 2, \ldots, n*m\}$ with $i \neq j$ uniformly at random, and exchanges just the operations located at those indices:

$$\mathcal{S}' = (\pi_1, \ldots, \underbrace{\pi_j}_{\text{pos } i}, \ldots, \underbrace{\pi_i}_{\text{pos } j}, \ldots, \pi_{n*m}) \tag{14}$$

This operator performs a local perturbation by swapping the execution priority of two operations, regardless of whether they belong to the same job or machine.

(b) **Shift Reinsertion Operator (SRO).** Removes an operation at a randomly selected position $i \in \{1, 2, \ldots, n*m\}$ from sequence $\mathcal{S}$ and reinserts it at a different random index $j \in \{1, 2, \ldots, n*m\} \backslash \{i\}$. Mathematically, if $i < j$, the modified sequence is given by:

$$\mathcal{S}' = (\pi_1, \ldots, \pi_{i-1}, \pi_{i+1}, \ldots, \pi_j, \pi_{\mathbf{i}}, \pi_{j+1}, \ldots, \pi_N) \tag{15}$$

Conversely, if $i > j$, the element π_i is placed immediately before π_j. Unlike simple swapping, reinsertion shifts an entire sub-sequence of operations, allowing deeper structural moves.

(c) **2-Opt Operator.** This operator is famously known for its use for the *Traveling Salesman Problem*. Selects two distinct cut points $i, j \in \{1, 2, \ldots, n * m\}$ with $i < j$ uniformly at random, and reverses the order of the sub-sequence enclosed between indices i and j inclusive:

$$S' = (\pi_1, \ldots, \pi_{i-1}, \pi_{\mathbf{j}}, \pi_{\mathbf{j-1}}, \ldots, \pi_{\mathbf{i+1}}, \pi_{\mathbf{i}}, \pi_{j+1}, \ldots, \pi_{n*m}) \tag{16}$$

The 2-Opt operator preserves the global schedule structure outside the window $[i, j]$ while completely inverting the priority ordering within it, enabling the solver to break out of complex local minima.

2.5 Mixed Integer Linear Programming

Finally, to get the optimal makespan for all the own generated *jb* instances a MILP model was implemented using the linear model proposed in Section 1.2 and solved to optimality with the CBC solver (Forrest and Lougee-Heimer 2005) of Python's PuLP (Mitchell, O'Sullivan, and Dunning 2011). In order to let the solver converge to the optimum, no time limit was applied.

3 Results & Discussion

The results for all the experiments performed, across all 68 instances, are presented in Tables 2 and 3. Gap columns represent the mean relative gap to the optimal (see Equation 8); RT columns indicate the mean runtime in seconds; and %SP columns contain the ratio between instances where a solution (not necessarily optimal) was found and the total instances tested. In this sense, the smaller the relative gap, the runtime, and the greater the found solutions ratio, the better the performance of the algorithm. The line plots presented across this section serve the purpose of showing how that metrics change as the size of the problems increase.

The runtime for this analysis was parsed using $\mathbf{RT}_{\text{final}} = \min(\mathbf{RT}_{measured}, 60)$, where $\mathbf{RT}_{measured}$ is the reported runtime that can be found in the logs (see Appendix A), as it also considers the extra time required by Python to clean up the memory after each execution (even if it was stopped due time limit), which is significant in width search methods like UCS or Weighted A* with a small weight.

Table 2: Results for all experiments over the *toy* instances and the summary across all instances.

Algorithm	Parameters	Toy instances			All instances		
		Gap	RT	%SF	Gap	RT	SF
uninformed-dfs	`None`	0.065	22.221	1	2.925	57.222	1.000
uninformed-ucs	`None`	0.682	24.159	0.600	4.614	57.365	0.044
heuristic-bs	`heuristic: CoBo; beam width: 100`	0.133	0.029	1.000	0.718	6.403	1.000
heuristic-bs	`heuristic: CoBo; beam width: 1000`	0.092	0.274	1.000	2.126	24.720	0.794
heuristic-wa	`heuristic: CoBo; weight: 1`	0.404	24.110	0.800	4.594	57.361	0.059
heuristic-wa	`heuristic: CoBo; weight: 100`	0.031	24.079	1.000	4.178	57.359	0.221
heuristic-wa	`heuristic: MWRM; weight: 1`	0.404	24.128	0.800	4.594	57.362	0.059
heuristic-wa	`heuristic: MWRM; weight: 100`	0.033	24.100	1.000	3.898	57.360	0.294
local-sa	`neighbor: RSO; c. rate: 0.995`	0.000	0.015	1.000	0.071	0.073	1.000
local-sa	`neighbor: RSO; c. rate: 0.999`	0.000	0.068	1.000	0.033	0.336	1.000
local-sa	`neighbor: SRO; c. rate: 0.995`	0.000	0.023	1.000	0.062	0.075	1.000
local-sa	`neighbor: SRO; c. rate: 0.999`	0.000	0.067	1.000	0.030	0.337	1.000
local-sa	`neighbor: 2-Opt; c. rate: 0.995`	0.000	0.018	1.000	0.104	0.073	1.000
local-sa	`neighbor: 2-Opt; c. rate: 0.999`	0.000	0.074	1.000	0.064	0.351	1.000

Figure 1 shows the performance of each experiment across all problem sizes. It can be seen that, as the size of the instances grow (from the left to the right), some algorithms as Beam Search and Weighted A* are displaced to the left (lower rate of solutions found) because the width of its explorations becomes unmanageable to reach a feasible solution on time. On the other hand, all algorithms experiment an upward displacement, as it becomes harder to find quality solutions over a fixed time limit. The size of the dots (logarithmically proportional to the mean runtime) also tends to grow. We can say that the experiments that show greater resistance to this trend, are the most promising to scalate well.

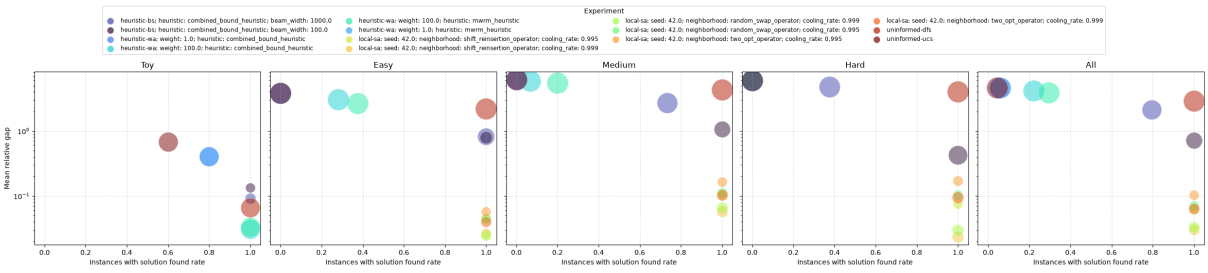


Figure 1: All experiments overall performance over different instance categories.

Table 3: Results for all experiments over *easy*, *medium* and *hard* instances.

Algorithm	Parameters	Easy instances			Medium instances			Hard instances		
		Gap	RT	SF	Gap	RT	SF	Gap	RT	SF
uninformed-dfs	None	2.184	60.000	1.000	4.284	60.000	1.000	4.027	60.000	1.000
uninformed-ucs	None	3.799	60.000	0.000	6.157	60.000	0.000	6.025	60.000	0.000
heuristic-bs	heuristic: CoBo; beam width: 100	0.795	0.906	1.000	1.058	5.920	1.000	0.426	19.842	1.000
heuristic-bs	heuristic: CoBo; beam width: 1000	0.830	8.374	1.000	2.692	38.520	0.733	4.823	52.113	0.375
heuristic-wa	heuristic: CoBo; weight: 1	3.799	60.000	0.000	6.157	60.000	0.000	6.025	60.000	0.000
heuristic-wa	heuristic: CoBo; weight: 100	3.076	60.000	0.281	5.941	60.000	0.067	6.025	60.000	0.000
heuristic-wa	heuristic: MWRM; weight: 1	3.799	60.000	0.000	6.157	60.000	0.000	6.025	60.000	0.000
heuristic-wa	heuristic: MWRM; weight: 100	2.677	60.000	0.375	5.524	60.000	0.200	6.025	60.000	0.000
local-sa	neighbor: RSO; c. rate: 0.995	0.045	0.047	1.000	0.113	0.095	1.000	0.104	0.121	1.000
local-sa	neighbor: RSO; c. rate: 0.999	0.025	0.216	1.000	0.066	0.440	1.000	0.030	0.562	1.000
local-sa	neighbor: SRO; c. rate: 0.995	0.045	0.050	1.000	0.104	0.096	1.000	0.076	0.120	1.000
local-sa	neighbor: SRO; c. rate: 0.999	0.026	0.215	1.000	0.057	0.444	1.000	0.023	0.565	1.000
local-sa	neighbor: 2-Opt; c. rate: 0.995	0.057	0.049	1.000	0.165	0.094	1.000	0.171	0.121	1.000
local-sa	neighbor: 2-Opt; c. rate: 0.999	0.040	0.229	1.000	0.104	0.459	1.000	0.094	0.581	1.000

In the following subsections we will review the results of each kind of algorithms in order to understand its generalization capabilities on bigger instances and the effect of its parameters on the quality of the found solutions.

3.1 Uninformed Search

Figure 2 shows the performance of the uninformed search algorithms (DFS and UCS) over different problem instance sizes. First of all, notice that the rate of solved instances varies radically between both algorithms. DFS always founds at least one feasible solution, as its depth-first nature prioritizes reaching the leaves of the tree (that is, to complete a schedule), even if it not the best. On the other hand, UCS, with its width-first nature, will not reach its first leaf until almost the rest of the tree is already explored. This guarantees that the first leaf reached is the optimal schedule, but if the hole tree cannot be explored within the time limit, UCS will not find any feasible solution (i.e. a complete schedule). That is the reason behind the zero solved instances rate for UCS on easy, medium and hard instances.

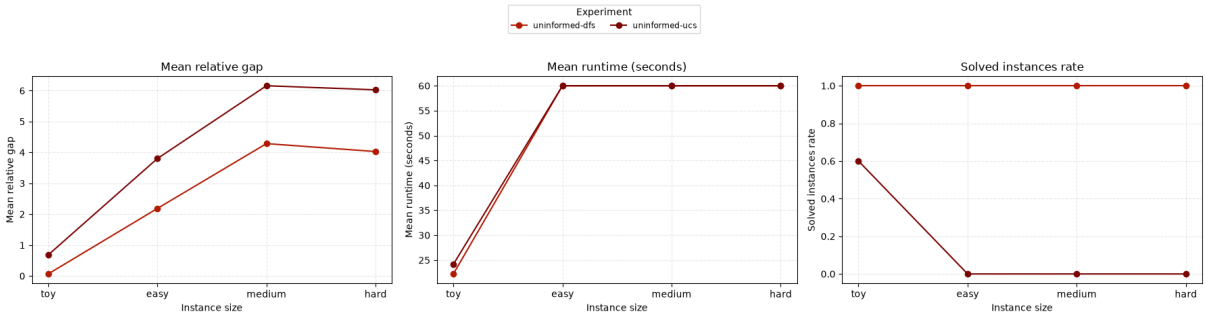


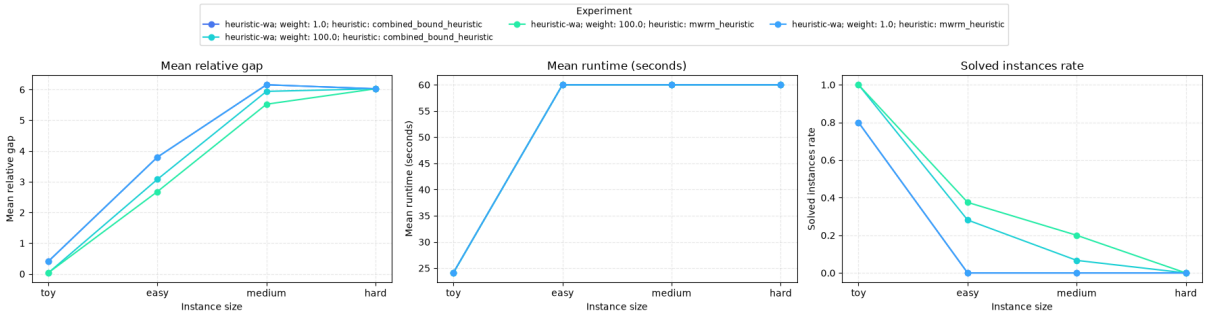
Figure 2: Uninformed search algorithms performance over different instance categories.

Even when there are no solutions found, there's still a makespan (and a gap) reported by UCS, which is a naive lower-bound (as explained in Section 2). It can be seen that the solutions found by DFS are closer to the optimal than the lower-bound reported by UCS for easy, medium, and hard instances. As the size of the instances grow, the relative gap of the DFS solution and the lower-bound also grows. DFS can guarantee optimality when it explores the hole tree, which does for all toy instances, but is

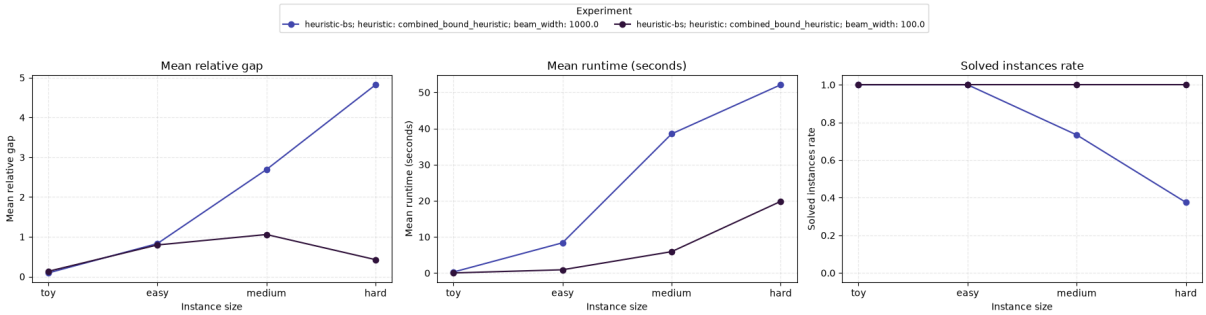
not achievable in time as the instance size grows. This is also clear in the plots: The mean runtime for non-toy instances is 60 seconds i.e. the exploration of all instance's trees was stopped by the time limit.

3.2 Heuristic Search

Two different heuristic search algorithms, and two different heuristic functions were implemented. First of all, Weighted A* showed important difficulties to scale to bigger instance sizes (see Figure 3a), given it's width-first nature. Relaxing this, and encouraging a greedy behavior by incrementing the weight of the heuristic function showed a significant improvement in the capacity to found solutions in bigger instance sizes. Also, MWRM heuristic showed best scaling capabilities as it is faster to compute that CoBo. On the other hand, it is difficult to provide a fair empiric comparison between the quality of solutions obtained with the two heuristic functions with this methodology, as the time limit is fixed and it allows simpler heuristics as MWRM to expand more nodes (i.e. to explore more possible solutions).



(a) Weigthed A*.



(b) Beam Search.

Figure 3: Heuristic search algorithms performance over different instance categories.

Beam search in an alternative to surpass the time limitation that affects Weighted A* as it restricts the growing of the frontier size, allowing reaching feasible solution in shorter times. Figure 3b shows the performance of the two variations of Beam Search using the CoBo heuristic function: One has `beam width:100` and the other one has `beam width:1000`. Restricting the beam width is key to preserve scalability. For this set of instances, `beam width:100` is small enough to complete its executions within the time limit.

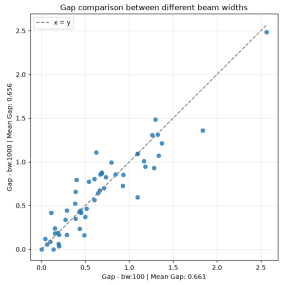


Figure 4: Relative gap comparison between two different beam widths.

In theory, reducing the beam width may result in a reduction of the quality of the output solutions as some promising solutions might be let out of scope in early stages of the search (i.e is not complete), but in this experiments that was not the case. Figure 4 shows that the quality of the solutions, in the instances where both configurations found one, is almost the same in terms of relative gap.

Said that, a beam search with a restricted beam width seems to be one of the most promising algorithms to get good JSSP solutions (7.18% mean relative gap) across different instance sizes in this work.

3.3 Local Search

The simulated annealing local search algorithm was executed with six variations: Three different neighborhood operators combined with two different cooling rates (leaving initial and minimal temperature fixed). Figure 5 shows the performance of this experiments. Notice that all of them were able to finish its executions within the time limit, as the number of steps of the algorithm depends of the temperature parameters more than the size of the instance. Even with that, the size of the instances have an impact in the mean runtime, as complexity of the evaluation of candidate solutions and the application of neighborhood operators depend of the instance size. Also, the cooling rate determines significantly the mean runtime of the algorithm. We can see in the central plot how the experiments group its runtimes in different ranges according to the cooling rate parameter.

In general, experiments with `cooling rate:0.999` reached solutions with lower relative gap, as they slow down the cooling process allowing the exploration of a larger portion of the solution space. Also, distinct neighborhood operators showed a consistent domination schema between both cooling rates: SRO obtained the lowest relative gaps across all instance sizes, closely followed by RSO, leaving 2-Opt behind. This makes sense as SRO and RSO preserve a better notion of locality because they make only small changes in the schedule, which ideally translates in small changes in the makespan. On the other hand, 2-Opt changes drastically the schedule reversing a subsequence, allowing big makespan changes between neighbors.

Simulated annealing with `cooling rate:0.999` and using SRO showed the best local search results across all instance sizes, with a 3% mean relative gap.

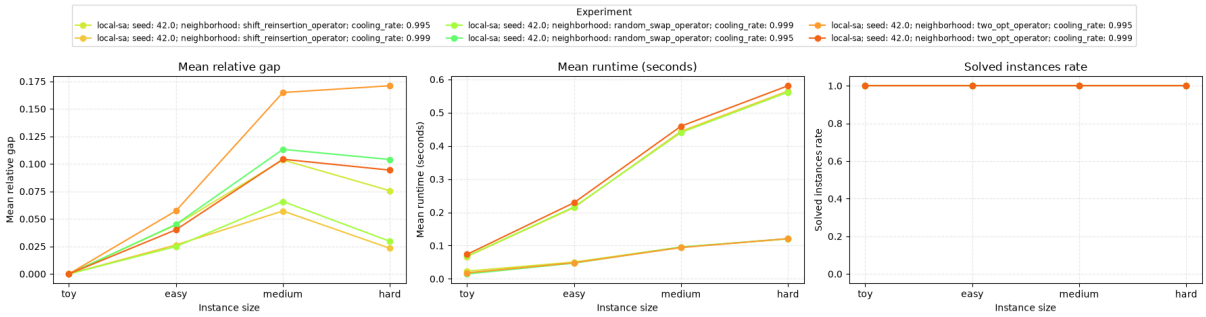


Figure 5: Local search with simulated annealing performance over different instance categories.

3.4 Mixed Integer Linear Programming

MILP formulation, solved with CBC allows to solve JSSP instances to optimality. In the case of this work we used it only for benchmarking, to solve the own generated instances (*toy* and *easy*) to optimality, in reasonable times (less than 5 hours per instance), but this was not scalable to *medium-hard* instances. So 15 extra own generated instances were left out of the result analysis.

4 Conclusion

Although JSSP can be modeled in ways that allow the application of uninformed, heuristic, and local search algorithms, the results obtained with them vary widely. In general, width-first algorithms become unusable for JSSP beyond toy instances, as they can not find any feasible solution in time. DFS will also lack of time to complete the hole tree exploration, but it will always find some feasible solutions in a short time, which can be used as initial solutions for local search algorithms, for example.

A regular Weighted A* (`weight:1`) suffers the same problems that UCS in terms of scalability, but they can be mitigated by highly increasing the weight of the heuristic function, or by limiting its frontier size (with Beam Search, for example). Under this time limit schema, the simpler heuristic functions reach best performances as they are allowed to explore more nodes. A Beam Search with a restricted beam width can finish its execution in restricted wall-clock times and get good quality solutions.

Local search with Simulated Annealing is also capable of finding good quality solutions within the time limit. A smaller cooling rate allows the algorithm a broader exploration of the solution space. And a reasonable neighborhood operator that preserves a notion of locality, not just in the state space but also for the value of the makespan, can improve the results of the search. Simulated annealing with `cooling rate:0.999` and SRO as neighborhood operator showed the best performance of all experiments across all instance sizes, with a 3% mean relative gap.

The time limit schema obstructs an in-depth comparison of certain algorithmic components, as it is not fair in all cases. This restriction was necessary to make all the algorithms computable in reasonable wall-clock time, due to the deliver dates, but no time restrictions are mandatory in order to reach a deeper analysis and results, as well as more experimentation with algorithm parameters.

References

[1] Balas, Egon (1969). "Machine Sequencing via Disjunctive Graphs: An Implicit Enumeration Algorithm". In: *Operations Research* 17.6, pp. 941–957. ISSN: 0030364X, 15265463. URL: http://www.jstor.org/stable/168317 (visited on 10/21/2026).

[2] Fisher, H and G Thompson (1963). "Probabilistic learning combination of local job-shop scheduling rules". In: *Industrial Scheduling. Prentice-Hall, Englewood Cliffs*.

[3] Forrest, John and Robin Lougee-Heimer (2005). "CBC User Guide". In: *Emerging Theory, Methods, and Applications*. INFORMS, pp. 257–277. DOI: 10.1287/educ.1053.0020.

[4] Google (2015). *6.1. The Job-Shop Problem, the disjunctive model and benchmark data*. Google OR-Tools. URL: https://acrogenesis.com/or-tools/documentation/user_manual/manual/ls/jobshop_def_data.html (visited on 10/21/2026).

[5] Lawrence, Stephen (1984). "Resouce constrained project scheduling: An experimental investigation of heuristic scheduling techniques (Supplement)". In: *Graduate School of Industrial Administration, Carnegie-Mellon University*.

[6] Manne, Alan (1960). "On the Job-Shop Scheduling Problem". In: *Operations Research* 8.2, pp. 219–223. URL: https://doi.org/10.1287/opre.8.2.219.

[7] Mitchell, Stuart, Michael O'Sullivan, and Iain Dunning (2011). "PuLP : A Linear Programming Toolkit for Python". In: URL: https://api.semanticscholar.org/CorpusID:14277904.

[8] Özgüven, Cemal, Lale Özbakır, and Yasemin Yavuz (2010). "Mathematical models for job-shop scheduling problems with routing and process plan flexibility". In: *Applied Mathematical Modelling* 34.6, pp. 1539–1548. ISSN: 0307-904X. DOI: https://doi.org/10.1016/j.apm.2009.09.002. URL: https://www.sciencedirect.com/science/article/pii/S0307904X09002819.

[9] ScheduleOpt (2024). *JSSP Benchmarks*. URL: https://github.com/ScheduleOpt/benchmarks.

[10] Valencia-Rivera, Gerardo Humberto et al. (2024). "A systematic review of metaheuristic algorithms in electric power systems optimization". In: *Applied Soft Computing* 150, p. 111047. ISSN: 1568-4946. DOI: https://doi.org/10.1016/j.asoc.2023.111047. URL: https://www.sciencedirect.com/science/article/pii/S1568494623010657.

[11] Wolpert, D.H. and W.G. Macready (1997). "No free lunch theorems for optimization". In: *IEEE Transactions on Evolutionary Computation* 1.1, pp. 67–82. DOI: 10.1109/4235.585893.

[12] Xiong, Hegen et al. (2022). "A survey of job shop scheduling problem: The types and models". In: *Computers & Operations Research* 142, p. 105731. ISSN: 0305-0548. DOI: https://doi.org/10.1016/j.cor.2022.105731. URL: https://www.sciencedirect.com/science/article/pii/S0305054822000338.

A GitHub Repository

All instances in JJSP format, algorithm implementations in python, execution logs, solutions found by each experiment, plots, notebooks for analysis, and every other artifact developed for this work are available in the following GitHub repository.

`https://github.com/juanjo-bltrn/tc6001-jssp`

B Classification of Instances by Difficulty Level

Just to introduce a notion of difficulty over all JSSP test instances, we define a difficulty level classification, based on the number of jobs (N) and the number of machines (M), according to Table 4. An instance is classified in the hardest category that contains it. In this sense, the *toy* problems can be used to test correctness of the implemented algorithms, as they are small enough to be hand-written, to explore all its solution space states in a short time, and to easily visualize its possible solutions. Schedules generated to solve *easy* problems can also be visualized to compare the performance of different algorithms, and *medium-hard* problems are expected to challenge the escalation capabilities of the algorithms.

Table 4: Distribution of instances over sets and difficulties.

Difficulty level	**Max. N**	**Max. M**
Toy	5	5
Easy	10	10
Medium	15	15
Hard	20	20

C System Specifications

To ensure computational reproducibility and enable valid performance comparisons across all evaluated algorithms, all experiments were conducted under the same hardware and software environment.

C.1 Hardware Environment

All benchmarks were performed on a device with the hardware configuration detailed in Table 5.

Table 5: Hardware system specifications.

Component	**Specification**
Host System Model	HP Envy x360 2-in-1 Laptop 16-ad0xxx
Processor (CPU)	AMD Ryzen 5 8640HS w/ Radeon 760M Graphics
Architecture	AMD Zen 4 (Hawk Point), 4 nm FinFET
Cores / Threads	6 Physical Cores / 12 Logic Threads
Clock Frequency	3.50 GHz (Base Clock) / up to 4.90 GHz (Boost)
Cache Memory	384 KB L1, 6 MB L2, 16 MB L3
System Memory (RAM)	8.00 GB DDR5 / LPDDR5 @ 6400 MT/s (7.29 GB usable)
Integrated GPU (iGPU)	AMD Radeon 760M Graphics (382 MB VRAM allocated)
Primary Storage	512 GB NVMe M.2 SSD (477 GB reported partition)
Instruction Set	x86_64 (64-bit)

C.2 Software Environment

The experimental codebase was executed within the software environment outlined in Table 6.

Table 6: Software platform and software dependencies.

Layer	Specification / Version
Operating System	Microsoft Windows 11 Home 64-bit
OS Build / Version	Version 25H2 (OS Build 26200.9711)
Experience Pack	Windows Feature Experience Pack 1000.26100.372.0
Runtime Environment	Python 3.13.7 (64-bit)
MILP Modeling Toolkit	PuLP (Python Linear Programming API)
Exact Solver	COIN-OR CBC (Branch and Cut) Solver
Core Libraries	`time`, `random`, `math`, `heapq`, `pulp`

C.3 Benchmarking Protocols & Conditions

To guarantee experimental control and comparability across all test instances:

- **Time Budget:** Each tested algorithm (DFS, UCS, Weighted A*, Beam Search, and Simulated Annealing) was constrained to a strict wall-clock timeout limit of $T_{\max} = 60.0$ seconds per instance. Exact MILP solver was run without time limits to convergence.
- **Time Measurement:** Execution time was measured using high-resolution CPU timing via Python's `time.perf_counter()`, recording elapsed wall-clock seconds (t_{exec}).
- **Stochastic Control:** Random seeds were explicitly fixed across all stochastic components (instance generator and simulated annealing neighborhood operators) to guarantee identical search trajectories upon re-execution.

D AI-Use Statement

- Tools used: Gemini 3.6, GitHub Copilot.
- What you used them for: Finding sources of information, debugging assistance, refactoring ideas, structure repository documentation.
- What was designed by your team (max 5 bullets): Abstraction and mathematical modeling of all algorithms and its components, methodology, code implementation, just to mention some things.
- How you verified correctness (max 5 bullets): Reasoning, I guess.
- Parts you could not defend without external help: None.